

GINZA GAMING · SECURITY REVIEW

# Ginza Gaming Forwarders.

A security review of Forwarders.

---

| CLIENT       | PROJECT    | AUDITORS | AUDIT WINDOW              | FINAL REPORT |
|--------------|------------|----------|---------------------------|--------------|
| Ginza Gaming | Forwarders | Guardian | April 23 – May 5,<br>2025 | May 12, 2025 |

§ 00

# Contents.

71 pages — overview, confidence ranking, scope, 53 findings, disclaimer, and about Guardian.

|      |                           |       |
|------|---------------------------|-------|
| § 01 | Overview                  | P. 03 |
| § 02 | Confidence Ranking        | P. 04 |
| § 03 | Scope & Classifications   | P. 05 |
|      | Files in Scope            | P. 06 |
| § 04 | Findings                  | P. 07 |
|      | Main Review · 53 findings | P. 11 |
| § 05 | Disclaimer                | P. 70 |
| § 06 | About Guardian            | P. 71 |

§ 01

# Overview.

Guardian reviewed Forwarders across a single review round. The final report includes 6 Critical, 6 High, 10 Medium, 31 Low findings.

|                        |                                                 |
|------------------------|-------------------------------------------------|
| AUDIT FIRM             | REPOSITORY                                      |
| Guardian               | GinzaGaming/solana-forwarder, GinzaGaming/ginza |
| CLIENT                 | METHODOLOGY                                     |
| Ginza Gaming           | Smart contract · Manual · Main Review           |
| PROJECT                | MAIN REVIEW COMMIT                              |
| Forwarders             | a53eeb2b71d8                                    |
| PREPARED BY            |                                                 |
| Guardian               |                                                 |
| AUDIT WINDOW           |                                                 |
| April 23 – May 5, 2025 |                                                 |
| FINAL REPORT           |                                                 |
| May 12, 2025           |                                                 |

## 1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

|                        |                   |                        |                    |                         |
|------------------------|-------------------|------------------------|--------------------|-------------------------|
| 1                      | 2                 | 3                      | 4                  | 5                       |
| VERY LOW<br>CONFIDENCE | LOW<br>CONFIDENCE | MODERATE<br>CONFIDENCE | HIGH<br>CONFIDENCE | VERY HIGH<br>CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY                     | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|------------------------------|-------|---------|-----|---------|----------|
| <span>■</span> Critical      | 6     | 0       | 6   | 0       | 0        |
| <span>■</span> High          | 6     | 0       | 6   | 0       | 0        |
| <span>■</span> Medium        | 10    | 0       | 10  | 0       | 0        |
| <span>■</span> Low           | 31    | 0       | 31  | 0       | 0        |
| <span>■</span> Informational | 0     | 0       | 0   | 0       | 0        |
| All severities               | 53    | 0       | 53  | 0       | 0        |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                         | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>              | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b>                   | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b>               | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b><br><b>ASSIGNED</b> | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b>               | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

§ 03

# Scope & Classifications.

Guardian reviewed Forwarders in `GinzaGaming/solana-forwarder`, `GinzaGaming/ginza`. The scope includes 52 source lines across 3 files.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

|                     |                                                                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| IMPACT · HIGH       | Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.                          |
| IMPACT · MEDIUM     | Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt. |
| IMPACT · LOW        | Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.               |
| LIKELIHOOD · HIGH   | Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.                 |
| LIKELIHOOD · MEDIUM | Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.                        |
| LIKELIHOOD · LOW    | Unlikely to ever occur in production.                                                                                       |

## Files in Scope.

| FILE                             | NSLOC | TOTAL |
|----------------------------------|-------|-------|
| onchain/src/GinzaComptroller.sol | 26    | 33    |
| onchain/src/ForwarderFactory.sol | 12    | 16    |
| onchain/src/Forwarder.sol        | 14    | 19    |

## § 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID          | TITLE                                          | CATEGORY       | SEVERITY | STATUS       | PAGE  |
|-------------|------------------------------------------------|----------------|----------|--------------|-------|
| MAIN REVIEW |                                                |                |          |              |       |
| C-01        | Unauthorized Deposit Can Block Future Deposits | Access Control | CRITICAL | ACKNOWLEDGED | p. 11 |
| C-02        | Lack of Treasury Address Check                 | Access Control | CRITICAL | ACKNOWLEDGED | p. 13 |
| C-03        | Lack of Check for Token Mint Address           | Validation     | CRITICAL | ACKNOWLEDGED | p. 14 |
| C-04        | Insecure logging design                        | Access Control | CRITICAL | ACKNOWLEDGED | p. 15 |
| C-05        | Users can specify arbitrary auditResult        | Validation     | CRITICAL | ACKNOWLEDGED | p. 17 |
| C-06        | Multiple access control weaknesses             | Access Control | CRITICAL | ACKNOWLEDGED | p. 19 |
| H-01        | System is Not Scalable                         | Access Control | HIGH     | ACKNOWLEDGED | p. 21 |
| H-02        | PNL withdrawal is blocked because of require   | Logical Error  | HIGH     | ACKNOWLEDGED | p. 22 |
| H-03        | Init DoS with pre-creation of Treasury ATA     | DoS            | HIGH     | ACKNOWLEDGED | p. 23 |
| H-04        | Cross-Chain Refunds Not Handled                | Logical Error  | HIGH     | ACKNOWLEDGED | p. 24 |
| H-05        | Backend Wallet Could Be Drained                | Griefing       | HIGH     | ACKNOWLEDGED | p. 25 |
| H-06        | Cashouts Double Count Topups                   | Logical Error  | HIGH     | ACKNOWLEDGED | p. 26 |
| M-01        | Config Update Leads To Unexpected Behavior     | Logical Error  | MEDIUM   | ACKNOWLEDGED | p. 27 |

## Findings continued.

| ID   | TITLE                                         | CATEGORY            | SEVERITY | STATUS       | PAGE  |
|------|-----------------------------------------------|---------------------|----------|--------------|-------|
| M-02 | Initialization Can Be Front-runned            | Access Control      | MEDIUM   | ACKNOWLEDGED | p. 28 |
| M-03 | Treasury can Receive funds in non-ATA Account | Unexpected Behavior | MEDIUM   | ACKNOWLEDGED | p. 29 |
| M-04 | State Divergence Risk on Solana Rollbacks     | Unexpected Behavior | MEDIUM   | ACKNOWLEDGED | p. 30 |
| M-05 | User Can Cause DoS Via Seed Collision         | DoS                 | MEDIUM   | ACKNOWLEDGED | p. 31 |
| M-06 | Encryption key stored alongside ciphertext    | Trust Assumptions   | MEDIUM   | ACKNOWLEDGED | p. 33 |
| M-07 | Unvalidated Emote content                     | Validation          | MEDIUM   | ACKNOWLEDGED | p. 34 |
| M-08 | Missing replay protection on ThirdWeb webhook | Configuration       | MEDIUM   | ACKNOWLEDGED | p. 36 |
| M-09 | Missing validation of smart contract calls    | Validation          | MEDIUM   | ACKNOWLEDGED | p. 37 |
| M-10 | Hardcoded limit of results in queries         | Configuration       | MEDIUM   | ACKNOWLEDGED | p. 38 |
| L-01 | Factory salt doesn't include the sender       | Warning             | LOW      | ACKNOWLEDGED | p. 39 |
| L-02 | Unsafe token transfers                        | Validation          | LOW      | ACKNOWLEDGED | p. 40 |
| L-03 | 0 amount flushing                             | Best Practices      | LOW      | ACKNOWLEDGED | p. 41 |
| L-04 | Unnecessary Accounts In update_config         | Informational       | LOW      | ACKNOWLEDGED | p. 42 |
| L-05 | Implementation Contract Can be Reinitialized  | Access Control      | LOW      | ACKNOWLEDGED | p. 43 |
| L-06 | Authority is not Transferrable                | Best Practices      | LOW      | ACKNOWLEDGED | p. 44 |
| L-07 | Use Of transfer Instead of transfer_checked   | Best Practices      | LOW      | ACKNOWLEDGED | p. 45 |
| L-08 | Token2022 Tokens are not Supported            | Best Practices      | LOW      | ACKNOWLEDGED | p. 46 |

## Findings continued.

| ID   | TITLE                                            | CATEGORY            | SEVERITY | STATUS       | PAGE  |
|------|--------------------------------------------------|---------------------|----------|--------------|-------|
| L-09 | Missing Validation to Destination Account        | Best Practices      | LOW      | ACKNOWLEDGED | p. 47 |
| L-10 | Unclear Web2 Integration with Solana             | Informational       | LOW      | ACKNOWLEDGED | p. 48 |
| L-11 | Contract Uses Outdated Solidity Version          | Best Practices      | LOW      | ACKNOWLEDGED | p. 49 |
| L-12 | Discrepancy In Withdraw Review Amount            | Warning             | LOW      | ACKNOWLEDGED | p. 50 |
| L-13 | tx_signature is not used                         | Best Practices      | LOW      | ACKNOWLEDGED | p. 51 |
| L-14 | Game States Are Not Truly Random                 | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 52 |
| L-15 | Incorrect account spaces                         | Best Practices      | LOW      | ACKNOWLEDGED | p. 53 |
| L-16 | Deposits can be processed by anyone              | Access Control      | LOW      | ACKNOWLEDGED | p. 54 |
| L-17 | ATA is not initialized on config update          | Informational       | LOW      | ACKNOWLEDGED | p. 55 |
| L-18 | EVM Withdrawals may fail                         | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 56 |
| L-19 | PnL is subtracted from total deposited           | Informational       | LOW      | ACKNOWLEDGED | p. 57 |
| L-20 | Missing toolchain version in Anchor.toml         | Best Practices      | LOW      | ACKNOWLEDGED | p. 58 |
| L-21 | Deposit Account Can Be Closed To Get Rent Refund | Best Practices      | LOW      | ACKNOWLEDGED | p. 59 |
| L-22 | Flushed DepositStatus Unused                     | Warning             | LOW      | ACKNOWLEDGED | p. 60 |
| L-23 | Unfinished development                           | Best Practices      | LOW      | ACKNOWLEDGED | p. 61 |
| L-24 | Cross-Chain Bridge Not Properly Supported        | Warning             | LOW      | ACKNOWLEDGED | p. 62 |
| L-25 | Lack of Address Validation                       | Best Practices      | LOW      | ACKNOWLEDGED | p. 63 |
| L-26 | retryWithdrawal Not Callable                     | Warning             | LOW      | ACKNOWLEDGED | p. 64 |

## Findings continued.

| ID   | TITLE                                   | CATEGORY          | SEVERITY | STATUS       | PAGE  |
|------|-----------------------------------------|-------------------|----------|--------------|-------|
| L-27 | Unvalidated patch of arbitrary gameData | Validation        | LOW      | ACKNOWLEDGED | p. 65 |
| L-28 | Unclear Function Naming                 | Best Practices    | LOW      | ACKNOWLEDGED | p. 66 |
| L-29 | Unhandled exceptions in webhook routes  | Configuration     | LOW      | ACKNOWLEDGED | p. 67 |
| L-30 | Missing HTTP Security Headers           | Configuration     | LOW      | ACKNOWLEDGED | p. 68 |
| L-31 | Lack of Server Side Validation          | Trust Assumptions | LOW      | ACKNOWLEDGED | p. 69 |

## CRITICAL C-01

ACKNOWLEDGED

# Unauthorized Deposit Can Block Future Deposits

CATEGORY Access Control  
LOCATION process\_deposit.rs  
REVIEW Main Review

## DESCRIPTION

The `process_deposit` function is currently permissionless, allowing anyone to call it on behalf of any user. Inside the function, the following operation is performed: `user.total_deposited += amount;`

An attacker could repeatedly call `process_deposit` for a target user, incrementing their `total_deposited` until it overflows to `u64::MAX`.

Once this happens, all future legitimate deposit attempts by the user would fail due to overflow checks, effectively locking the user out of the deposit flow.

Other issues include:

(1) User's `total_deposited` and `deposit_count` are updated immediately after creating a `Deposit` record. However, the actual token transfer from the user's wallet to the treasury occurs in a separate instruction. This decoupling allows malicious users to falsely inflate their on-chain deposit history without transferring any real tokens, potentially enabling fraudulent withdrawals.

(2) Arbitrary user can specify an invalid `tx_signature`, hence the draft Solana backend will retrieve an invalid last processed signature for deposit processing.

## RECOMMENDATION

Restrict `process_deposit` so that only Ginza's system can call it. Restricting `process_deposit` so that only the `user.authority` (i.e., the rightful owner of the user account) is allowed to call it is not enough, since the `tx_signature` can still be arbitrarily passed.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/solana-forwarder-team1/blob/2eca746e8495bc63a305f404e5151b47fdd8296d/tests/solana-forwarder.ts#L446-L527>

---

**C-01** Unauthorized Deposit Can Block Future Deposits

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

CRITICAL **C-02**

ACKNOWLEDGED

# Lack of Treasury Address Check

|          |                                                  |
|----------|--------------------------------------------------|
| CATEGORY | Access Control                                   |
| LOCATION | transfer_to_treasury.rs : L11 , withdraw.rs: L11 |
| REVIEW   | Main Review                                      |

## DESCRIPTION

There is currently no proper validation for the treasury account in either the transfer\_to\_treasury or withdraw instructions.

The only enforced constraint is:

```
treasury_token_account.owner == treasury.key()
```

This check is insufficient. A malicious user can create a random address, initialize a token account for that address, and pass both as `treasury` and `treasury_token_account` respectively. This bypasses the intended validation and allows two types of attacks:

- **In `transfer_to_treasury`:** The user's tokens are transferred from their account into an attacker-controlled token account (fake treasury). However, the deposit is still considered valid and processed by Ginza, despite no funds reaching the real treasury.
- **In `withdraw`:** An attacker can effectively delete another user's `total_deposited` value, causing disruption to users' withdrawal capabilities.

## RECOMMENDATION

Add a constraint to both `TransferToTreasury` and `Withdraw` instructions to validate that the provided `treasury` matches the protocol's configured treasury: `constraint = config.treasury == treasury`

To implement this, the `config` account (with its proper seeds) must be added as an additional account to these instructions.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/solana-forwarder-team2/commit/29afe112b31c14b214d558a7e254c3829aecde2d>

## RESOLUTION

Status: Acknowledged.

## CRITICAL C-03

ACKNOWLEDGED

# Lack of Check for Token Mint Address

CATEGORY Validation  
LOCATION transfer\_to\_treasury.rs  
REVIEW Main Review

## DESCRIPTION

The `transfer_to_treasury` instruction should transfer `USDC` from the user to the treasury. It then marks the given deposit as `processed`. However, the only validation performed on `user_token_account` and `treasury_token_account` is that their owners are respectively the user and the treasury. This allows the user to create a new bogus token to use as payment instead of USDC. In result, the deposit will be marked as processed and Ginza will reward the user, but the treasury will not receive any `USDC`. This will ultimately result in improper crediting and loss of protocol funds.

## RECOMMENDATION

Add the following constraint to the `treasury_token_account`: `constraint = treasury_token_account.mint == config.token_mint`

Additionally, ensure that the `config` account is provided as part of the `TransferToTreasury` instruction context so this constraint can be validated during execution.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/solana-forwarder-team1/blob/46cca58ad65ff18c380d8b342daa8e189f455c44/tests/solana-forwarder.ts#L582-L731>

## RESOLUTION

Status: Acknowledged.

CRITICAL **C-04**

ACKNOWLEDGED

# Insecure logging design

CATEGORY Access Control  
LOCATION Global  
REVIEW Main Review

## DESCRIPTION

First, the application stores cards data in logs in `apps/backend/src/model/games/poker/model.py`:

```
def deal_cards_to_board(self, num_cards: int, log_type: LogType) -> None:
    ...
    self.log(log_type, args=tuple(map(str, self.board)))
```

Unmasked board (and later, player hand) strings are written to logs. If an attacker gains log access, they can see private hands and gain an unbeatable advantage. In the same time there is an unauthenticated query in `tasks.ts`:

```
export const getGameLogs = query({
  args: { gameId: v.id("gameData") },
  handler: async (ctx, { gameId }) => {
    const gameLogs = await ctx.db
      .query("gameLogs")
      .withIndex("byGameId", (q) => q.eq("gameId", gameId))
      .order("desc")
      .take(250);
    return gameLogs.toReversed();
  },
});
```

This means that unauthenticated users are able to view other user's hands by viewing the logs where the cards are stored and gain an unfair advantage over other players.

Other than that, application logs should never be exposed to users, as they may contain other sensitive information utilized by the application.

## RECOMMENDATION

**C-04** Insecure logging design

CONTINUED

Any logs should not be accessible by unauthenticated users, including post game. Encrypt the logs containing sensitive information such as users' current cards to rule out possibility of someone gaining an unfair advantage.

**— RESOLUTION**

Status: Acknowledged.

## CRITICAL C-05

ACKNOWLEDGED

# Users can specify arbitrary auditResult

|          |                                       |
|----------|---------------------------------------|
| CATEGORY | Validation                            |
| LOCATION | packages/convex/convex/withdrawals.ts |
| REVIEW   | Main Review                           |

## DESCRIPTION

In function `requestWithdrawal` one of user-supplied argument is `auditResult`. Its value is later used to determine, whether status of withdrawal should be set to `PENDING` or `UNDER_REVIEW`. However that value comes from users, which means it can be arbitrarily set up to `PASSED` to potentially bypass application controls and avoid the review.

```
export const requestWithdrawal = userAuthenticatedMutation({
  args: {
    withdrawAddress: v.string(),
    withdrawAmount: v.number(),
    tokenContractAddress: v.string(),
    chainId: v.number(),
    auditResult: v.string(),
  },
  handler: async (ctx, { withdrawAddress, withdrawAmount, tokenContractAddress, chainId,
    auditResult }) => {
    [...]
    // Validate auditResult is one of the allowed values
    if (auditResult !== "PASSED" && auditResult !== "FAILED") {
      throw new Error("Invalid audit result. Must be 'PASSED' or 'FAILED'");
    }
    [...]
    // Determine status based on audit result AND amount threshold
    const status =
      auditResult === "FAILED" || withdrawAmount >= REVIEW_THRESHOLD_MICRODOLLARS
      ? "UNDER_REVIEW"
      : "PENDING";
```

## RECOMMENDATION

**C-05** Users can specify arbitrary auditResult

CONTINUED

The audit result should be generated by the system based on actual verification, not provided by the user. Remove this parameter from the user-facing API and add the check server side. Furthermore, ensure `withdrawAmount` so it is not negative.

**— RESOLUTION**

Status: Acknowledged.

CRITICAL **C-06**

ACKNOWLEDGED

# Multiple access control weaknesses

|          |                |
|----------|----------------|
| CATEGORY | Access Control |
| LOCATION | Global         |
| REVIEW   | Main Review    |

## DESCRIPTION

The authorization on application functions is implemented incorrectly, having many weak points that may allow unauthorized actors to invoke actions they should not have permission to. Most of the misconfigurations are caused by a fact that user A is able to view all data of user B, possibly revealing their private data or a game's private data.

In `withdrawals.ts`:

- `getWithdrawal` - Allows anyone to read any withdrawal record by ID
- `batchGetWithdrawalsWithUsers` - Exposes all withdrawal data along with user information
- `getUserPendingWithdrawals` - Allows reading any user's pending withdrawals

In `tasks.ts`:

- `getAccountBalance` - Allows anyone to view any account's balance by ID
- `getGameData` - Fetches game data without restrictions
- `getGameTransactionsForUser` - Anyone can view any user's game transactions
- `getGameTransactionsForGame` - Anyone can view any game's transactions
- `getUserById` - Allows anyone to fetch arbitrary user data by ID
- `getUserByUsername` - Allows anyone to fetch user data by username
- `getGameLogs` - Allows viewing any game's logs
- `getEncryptedDecks` - Exposes encrypted decks
- `getLifetimeRewardsForUser` - Allows viewing any user's lifetime rewards

In `tasks.ts`, a public mutation:

- `initDegenRouletteGame` - Allows anyone to initialize a degenerate roulette game
- `transferFunds` - Allows anyone to debit an arbitrary user.
- `updateGameData` - Allows anyone to set the game's data.
- `updateGameDataAndTimeout` - Allows anyone to set the game's data.
- `updateGameDataAndDeleteTimeone` - Allows anyone to set the game's data.

**C-06** Multiple access control weaknesses

CONTINUED

**— RECOMMENDATION**

Implement proper authorization mechanisms following the Principle of Least Privilege (every action is denied by default unless explicitly allowed). Restrict users from accessing sensitive data of other users.

**— RESOLUTION**

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

# System is Not Scalable

CATEGORY Access Control  
LOCATION GinzaComptroller.sol : L24  
REVIEW Main Review

## DESCRIPTION

The `batchFlushERC20` function iterates over all user accounts and calls each forwarder's `flushERC20` function. This approach is not scalable. Given Base chain's block gas limit of around 120 million gas, `batchFlushERC20` can only handle approximately 8,000 users before running into an out-of-gas (OOG) error. As the user base grows beyond this point, the function will fail and become unusable. This creates a critical bottleneck for scaling user deposits.

Furthermore, Convex's `processDeposits` function also operates on all users in the system, which may lead to performance issues and suboptimal memory consumption.

## RECOMMENDATION

Process batch functions in smaller portions, not to overload any depending function. This would ensure the system remains operational even as the number of users increases significantly.

PROOF OF CONCEPT <https://github.com/GuardianOrg/ginza-team2/pull/1>

## RESOLUTION

Status: Acknowledged.

<https://github.com/GinzaGaming/ginza/commit/303efcb96aa19125a340c05e76a006037ad8011f>

HIGH **H-02**

ACKNOWLEDGED

# PNL withdrawal is blocked because of require

CATEGORY Logical Error  
LOCATION withdraw.rs  
REVIEW Main Review

## — DESCRIPTION

When the `withdraw` instruction is executed, it's enforced that the amount requested is less than `user.total_deposited`. This will block any withdrawals where the user has positive PnL or any additional rewards since the amount they should be able to withdraw is more than what they deposited.

## — RECOMMENDATION

One option is to remove the `require` statement. However, this will allow the treasury (assuming only the treasury can call this instruction) to withdraw all the funds and send them to a certain user, which introduces a serious trust issue. You can come up with a mechanism that adjusts the `total_deposited` of users based on their PnL - both positive and negative. While this approach would not result in giving the treasury more power than it should have, it's more complex.

## — RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

# Init DoS with pre-creation of Treasury ATA

CATEGORY DoS  
LOCATION initialize.rs: L17  
REVIEW Main Review

## DESCRIPTION

The `treasury_token_account` is an Associated Token Account (ATA) with the mint set to `token_mint` (USDC) and authority set to `treasury`. Since both of these accounts are publicly known, any user can preemptively create the ATA for the treasury address.

This would cause a Denial of Service (DoS) for the `initialize` function because the `init` constraint expects the account to be uninitialized. If the ATA already exists, `initialize` will revert and the protocol will be unable to complete setup.

## RECOMMENDATION

Change the `init` constraint to `init_if_needed` for `treasury_token_account` in `initialize`. This will allow the function to continue even if the ATA has already been created.

PROOF OF CONCEPT <https://github.com/GuardianOrg/solana-forwarder-team2/commit/5ae44a69b4f199f8c8dae2810c52ee73a769c070>

## RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

# Cross-Chain Refunds Not Handled

CATEGORY Logical Error  
LOCATION withdrawals.ts  
REVIEW Main Review

## — DESCRIPTION

According to Relay documentation, "a refund is returned funds to a user due to a failed swap or bridge. Relay will refund the user's their funds on the destination chain. If the destination chain is unavailable, Relay will refund the user's funds on the deposit chain. The destination chain might be unavailable for a variety of reasons like a network outage."

Ginza's existing cross-chain logic is not capable of handling refunds as it assumes the bridge will always be successful. Consequently user withdrawals will be processed yet they will not receive their funds in the case of a Relay refund, leading to loss of user funds.

## — RECOMMENDATION

Enforce handling for Relay funds by passing to the Relay API `refundTo` and `refundOnOrigin`. This will allow Ginza to control a specific refund address which can be used to process refunds and allow for user funds distribution in the case of a failed bridge.

## — RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

# Backend Wallet Could Be Drained

CATEGORY Griefing  
LOCATION Global  
REVIEW Main Review

## DESCRIPTION

The `BACKEND_WALLET_1` is responsible for covering the execution cost of all on-chain interactions within the function `contractWrite`.

Consequently, this exposes risk for the protocol since a user can spam withdrawal requests in order to increase the costs on the protocol.

Consider the following scenario:

- (1) Bob wants to withdraw 100
- (2) Instead of requesting a withdrawal for 100 and the `BACKEND_WALLET_1` only having to call on-chain `processWithdrawal` once, Bob requests a withdrawal of 1, but 100 times.
- (3) `processWithdrawal` will have to be called 100 times to perform the transfer, exacerbating the gas cost the `BACKEND_WALLET_1` has to cover.

## RECOMMENDATION

Implement a minimum withdrawal amount as a % of total balance to prevent spam-like requests. Furthermore, consider implementing rate-limiting.

## RESOLUTION

Status: Acknowledged.

HIGH **H-06**

ACKNOWLEDGED

# Cashouts Double Count Topups

CATEGORY Logical Error  
LOCATION poker.ts  
REVIEW Main Review

## DESCRIPTION

Function `top_up_player` is used to add a top-up for the player but also appends a transfer to `transfers_to_process`.

After a play action, the game data is updated and the top-up transfer within `transfers_to_process` is processed. However, the newly appended `game_data.top_ups` entry still remains until the current hand is reset. This allows for the following attack vector:

- (1) Bob triggers `top_up_player` with amount 100
- (2) Bob then calls `cashout_player` so he is in the `players_to_cashout` list
- (3) A player in the game makes an action, which updates the game data and Bob's transfer is processed with `maybeProcessTransfers`. Now Bob's Player `amount` reflects the 100.
- (4) Hand is reset and `processRewardsAndCashouts` is triggered. Bob receives not only the 100 that is still within `gameData.top_ups`, but also the 100 that his latest balance reflects. Consequently, Bob is able to receive 200 (twice the initial topup) for a single cashout.

## RECOMMENDATION

Clear the `gameData.top_ups` when the top-up transfer is processed in the ledger.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

# Config Update Leads To Unexpected Behavior

CATEGORY Logical Error  
LOCATION lib.rs : L49  
REVIEW Main Review

## DESCRIPTION

The `update_config` function allows updating the `treasury` and `token_mint` accounts used across the system.

However, user deposits are stored in `user.total_deposited` based on the old `token_mint`'s denomination, without any conversion to a stable USD-equivalent value.

This creates serious inconsistencies if the `token_mint` is changed:

- If the new token is more valuable than the old one, users could withdraw more value than they deposited, resulting in a loss to the protocol.
- If the new token is less valuable, users would withdraw less value than they deposited, causing user fund loss.
- If the new token has different decimals from the old token, it could cause even more severe issues including over/under payment

Thus, changing the `token_mint` would cause unexpected and critical accounting errors.

## RECOMMENDATION

Change of `token_mint` should not be permitted.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

# Initialization Can Be Front-runned

CATEGORY Access Control  
LOCATION initialize.rs: L9  
REVIEW Main Review

## — DESCRIPTION

Deployment of the Ginza program and the call to `initialize` on Solana are not atomic — meaning the program is first deployed, and `initialize` must be called in a separate transaction.

Since `initialize` lacks any access control, anyone can call it immediately after deployment. A malicious actor could front-run the intended initializer, set the `config` authority to their own address, and take full control of the protocol's configuration and operations.

## — RECOMMENDATION

- Deploy and call `initialize` in the same transaction using a deployment script.
- Alternatively, if the intended authority is known before deployment, hardcode the expected authority address inside the `initialize` function to ensure only that address can call it.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

# Treasury can Receive funds in non-ATA Account

CATEGORY Unexpected Behavior  
LOCATION `transfer_to_treasury.rs` : L15  
REVIEW Main Review

## DESCRIPTION

In `transfer_to_treasury`, users can use a token account that is not ATA for treasury. This can be done by creating a random token account for `treasury` and using that account to transfer.

While these funds would be retrievable by `treasury`, it can create lack of funds issues for withdrawals for some time (until it is noticed and handled) considering treasury ATA would be used for withdrawals in off-chain system.

## RECOMMENDATION

Change constraint in the `transfer_to_treasury` s `treasury_token_account` with:  
`associated_token::authority = treasury.key()`

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

# State Divergence Risk on Solana Rollbacks

|          |                     |
|----------|---------------------|
| CATEGORY | Unexpected Behavior |
| LOCATION | General             |
| REVIEW   | Main Review         |

## — DESCRIPTION

In the event of a Solana rollback, certain deposits and withdrawals may be reverted on-chain. However, since these actions are processed through the off-chain backend system, inconsistencies between on-chain and off-chain states can occur.

This discrepancy can lead to potential fund loss either for users or for the protocol.

For example, a deposit that is rolled back on-chain could still be recorded as successful off-chain, allowing the user to play with funds that the treasury never actually received.

## — RECOMMENDATION

Implement robust reconciliation logic that periodically compares on-chain state with off-chain records. Any inconsistencies should be flagged for manual review or automatically halted until resolved.

Additionally, delay crediting or unlocking funds off-chain until a sufficient number of Solana block confirmations are observed to mitigate rollback risks.

`LastRestartSlot` sysvar can be utilized to check latest rollback.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

# User Can Cause DoS Via Seed Collision

CATEGORY DoS  
LOCATION create\_user.rs:L13  
REVIEW Main Review

## DESCRIPTION

The `create_user` function allows anyone to create a user account by specifying a `player_id`. However, the function is permissionless and does not validate ownership of the `player_id`.

This opens up a vulnerability where an attacker can front-run and register a `player_id` that already exists in the backend database (e.g., a player created via the EVM flow).

This can lead to the following issues:

- The attacker could potentially hijack another user's `player_id`, possibly allowing them to withdraw funds or interfere with that user's game interactions.
- The rightful user would be blocked from using the Solana flow for deposits or withdrawals, as their player ID would already be "taken" on-chain.
- Malicious users can grief `User` account creation by using a common or targeted `player_id`. Once created, any subsequent attempts to create a new `User` account with the same `player_id`—regardless of the rightful owner—will fail due to PDA address collision.

Example Attack Scenario:

- Alice signs up to Ginza with `player_id = 1` which is created via EVM flow.
- Through the Solana flow `create_user(1)` for Alice is called.
- Bob front-runs and calls `create_user(1)` first.
- Bob now owns `player_id 1` on Solana and could potentially interact with Alice's game data or steal her funds.
- Alice is locked out from using her rightful account via Solana

## RECOMMENDATION

Restrict the `create_user` function so that only an approved Keeper address (trusted backend actor) can call it.

---

**M-05** User Can Cause DoS Via Seed Collision

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

# Encryption key stored alongside ciphertext

**CATEGORY** Trust Assumptions**LOCATION** packages/convex/convex/schema.ts – definition of the `encryptedDecks` table, packages/convex/convex/tasks.ts – `saveEncryptedDeck` mutation**REVIEW** Main Review

## — DESCRIPTION

The `encryptedDecks` table persists both the encrypted deck blob ( `encryptedDeck` ) and its decryption key ( `encryptionKey` ) in the same record. Likewise, the `saveEncryptedDeck` mutation writes both values without separation.

This is problematic, as any actor with read privileges on the database can fetch the key alongside the ciphertext and trivially decrypt every deck, completely voiding the purpose of encryption. This can happen also during, for example, SQL-injection or similar types of attacks.

## — RECOMMENDATION

We recommend moving all encryption keys out of the application database into a dedicated secrets management system. Store only the ciphertext in your DB, and inject decryption keys into the game server at runtime under strict access control.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

# Unvalidated Emote content

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Validation                      |
| LOCATION | packages/convex/convex/tasks.ts |
| REVIEW   | Main Review                     |

## DESCRIPTION

The sendEmote function in task.ts accepts user-provided string content without implementing any validation controls. This function allows authenticated users to submit arbitrary string content as "emotes" which are stored in the database and potentially displayed to other users.

```
export const sendEmote = userAuthenticatedMutation({
  args: {
    gameId: v.id("gameData"),
    emote: v.string(),
  },
  handler: async (ctx, { gameId, emote }) => {
    await ctx.db.insert("emotes", {
      gameId,
      userId: ctx.userId,
      emote,
    });
  },
});
```

The emote parameter accepts any string value without validation for content type, length restrictions, or format requirements. The function lacks validation to confirm the target gameId exists or that the user has appropriate permissions to interact with that specific game. Furthermore, the lack of game validation could allow users to send emotes to games they aren't participating in or to non-existent game sessions, potentially causing data integrity issues or enabling cross-game communication vectors unintended by the application design. The system stores unfiltered user content that could contain offensive material, XSS payloads, malware links, or social engineering content that might be displayed to other players, potentially damaging user experience and platform reputation.

---

**M-07** Unvalidated Emote contentCONTINUED

---

**— RECOMMENDATION**

Implement complex input validation for the emote parameter, restrict emote content to a predefined whitelist of acceptable emotes or patterns. Moreover, validate that the target game exists and that the user has permission to interact with it.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

# Missing replay protection on ThirdWeb webhook

CATEGORY Configuration

LOCATION packages/convex/convex/webhook\_validators.ts - `validateThirdWebWebhook`

REVIEW Main Review

## DESCRIPTION

Although the HMAC signature is verified in the code logic and requests, the `validateThirdWebWebhook` function never checks the age of the `X-Engine-Timestamp` header.

Without rejecting timestamps outside a small tolerance, an attacker who captures a valid webhook payload can replay it indefinitely. Additionally, even through implementation of some toleration - fast replays might be still possible, as system is not implementing nonces, id's or other protection for this specific attack.

Ref: <https://www.svix.com/guides/receiving/receive-webhooks-with-svix-cli/>

## RECOMMENDATION

Before signature verification, parse `X-Engine-Timestamp` as a UNIX timestamp and reject any request older than a configurable window or outside tolerated clock skew. Additionally, if replay protection risk have to be eliminated fully - extra mechanism in a form of the nonce or id's should be implemented, to disallow non-atomic executions of the same signature:timestamp pairs.

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

# Missing validation of smart contract calls

|          |                                   |
|----------|-----------------------------------|
| CATEGORY | Validation                        |
| LOCATION | packages/convex/convex/onchain.ts |
| REVIEW   | Main Review                       |

## — DESCRIPTION

The `contractWrite` helper simply returns the `queueId` from the ThirdWeb Engine, without ever checking that the transaction actually succeeded on-chain.

A `queueId` only means “we’ve accepted your request for processing,” not that it was mined, not that it didn’t revert, or that any state changes (or return values/events) occurred as intended.

Silent failures or reverts will be missed entirely, leading the backend to assume success even when the on-chain call failed.

## — RECOMMENDATION

After calling `engine.contract.write(...)`, use the Engine/ThirdWeb API to fetch or poll for the transaction receipt (by `queueId` or returned tx hash) and verify that the receipt’s status is “success”.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

# Hardcoded limit of results in queries

|          |                          |
|----------|--------------------------|
| CATEGORY | Configuration            |
| LOCATION | packages/convex/convex/* |
| REVIEW   | Main Review              |

## DESCRIPTION

Multiple endpoints in the application use hardcoded result limits when querying the database. This pattern can lead to data incompleteness where not all relevant records are returned to the client. Moreover, it is possible for an attacker to maliciously store as much data as possible, overwriting legitimate data and denying access to it.

For example, this is `getEmotes` in `tasks.js`:

```
export const getEmotes = query({
  args: { gameId: v.id("gameData") },
  handler: async (ctx, { gameId }) => {
    return await ctx.db
      .query("emotes")
      .withIndex("byGameId", (q) => q.eq("gameId", gameId))
      .order("desc")
      .take(25);
  },
});
```

Other functions are:

- `tasks.ts::getGameLogs`
- `withdrawals.ts::batchGetWithdrawalsWithUsers` and `getUnderReviewWithdrawalsWithUsers`
- `deposits.ts::batchGetDepositsWithUsers`

## RECOMMENDATION

Allow users to query a specific range of records instead of always returning a hardcoded amount.

## RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

## Factory salt doesn't include the sender

|          |                          |
|----------|--------------------------|
| CATEGORY | Warning                  |
| LOCATION | ForwarderFactory.sol:L16 |
| REVIEW   | Main Review              |

### — DESCRIPTION

The deployment of the contract in `initForwarder` doesn't include the `msg.sender` in the salt. This means anyone can call the function and deploy a forwarder for any user. This will result in reverting of all subsequent calls to that function with the same salt, which may be unexpected behavior depending on how it's used onchain.

### — RECOMMENDATION

Consider whether adding `msg.sender` towards the salt is desirable. Otherwise, implement access control on `initForwarder`, allowing only approved addresses like the Keeper to call it.

### — RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

# Unsafe token transfers

|          |                                     |
|----------|-------------------------------------|
| CATEGORY | Validation                          |
| LOCATION | GinzaComptroller.sol; Forwarder.sol |
| REVIEW   | Main Review                         |

## DESCRIPTION

The `GinzaComptroller` and `Forwarder` contract both use `ERC20.transfer()`, but some tokens don't revert on failed transfers - instead they just return `false`.

## RECOMMENDATION

If you are willing to support such tokens, implement `SafeERC20`.

## RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

## 0 amount flushing

CATEGORY Best Practices  
LOCATION Forwarder.sol:L17  
REVIEW Main Review

### — DESCRIPTION

The `Forwarder.flushERC20()` function initiates token transfer even if the current balance for this token is 0. This can result in the transaction reverting if the token doesn't support 0 amount transfers and in result cause a revert for `GinzaComptroller.batchFlushERC20()`.

### — RECOMMENDATION

Execute the transfer only if the balance of the contract is positive.

### — RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

# Unnecessary Accounts In update\_config

CATEGORY Informational  
LOCATION update\_config.rs  
REVIEW Main Review

## — DESCRIPTION

In the `update_config` instruction, following accounts and programs are not used:

- `treasury_token_account`
- `system_program`
- `token_program`
- `associated_token_program`
- `rent`

This introduces unnecessary overhead by requiring extra accounts during the instruction call and also increases call's stack size with unnecessary programs.

## — RECOMMENDATION

Remove unused accounts from the `update_config` instruction.

## — RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

# Implementation Contract Can be Reinitialized

CATEGORY Access Control  
LOCATION GinzaComptroller.sol  
REVIEW Main Review

## DESCRIPTION

The `GinzaComptroller` contract inherits from `UUPSUpgradeable` and includes an `initialize` function to configure roles. However, it fails to call `_disableInitializers()` in the contract's constructor. As a result, the implementation contract remains unprotected and callable. A malicious user could directly invoke `initialize` on the implementation contract to gain admin roles.

## RECOMMENDATION

Call `_disableInitializers()` in the constructor to prevent reinitialization of the implementation contract.

```
+ constructor() {  
+   _disableInitializers();  
+ }
```

## RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

# Authority is not Transferrable

|          |                |
|----------|----------------|
| CATEGORY | Best Practices |
| LOCATION | General        |
| REVIEW   | Main Review    |

## — DESCRIPTION

The `config.authority` is not transferable.

If the authority's relationship with the protocol is ever severed (e.g., loss of keys, organizational changes), there would be no way to update critical parameters such as the `treasury` address or `token_mint`.

This would permanently lock the protocol's ability to perform essential administrative actions.

## — RECOMMENDATION

Implement a two-step authority transfer mechanism for `config`, where the current authority initiates the transfer and the new authority must explicitly accept it.

## — RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

# Use Of transfer Instead of transfer\_checked

|          |                                            |
|----------|--------------------------------------------|
| CATEGORY | Best Practices                             |
| LOCATION | TransferToTreasury.rs:L53, withdraw.rs:L50 |
| REVIEW   | Main Review                                |

## DESCRIPTION

The `TransferToTreasury` and `withdraw` handler use the `transfer` instruction to move tokens from a user's payment account to the treasury payment account. However, `transfer` does not validate the token mint or decimal configuration, allowing transfers involving unintended token types or incorrect amounts due to misinterpreted decimals. Without enforcing these checks, users or integrators may inadvertently or maliciously trigger transfers under misconfigured conditions.

## RECOMMENDATION

Replace all `transfer` calls with `transfer_checked`, which validates both the mint and the number of decimals.

```
- transfer(...)
+ transfer_checked(...,)
```

## RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

# Token2022 Tokens are not Supported

CATEGORY Best Practices  
LOCATION General  
REVIEW Main Review

## — DESCRIPTION

Although the initial design expects only USDC (an SPL token) for deposits on Solana, the `update_config` instruction allows changing the accepted token.

If, in the future, support for Token2022 tokens is introduced, the current implementation will fail because it assumes SPL token behavior.

## — RECOMMENDATION

If Token2022 support is planned, update the implementation by:

- Using `InterfaceAccount` instead of `Account` for token mints.
- Replacing `transfer` calls with `transfer_checked` to properly handle extended token functionalities.

If no support for Token2022 is intended, then explicitly document this limitation.

## — RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

# Missing Validation to Destination Account

CATEGORY Best Practices

LOCATION withdraw.rs:L20

REVIEW Main Review

## — DESCRIPTION

In the `Withdraw` instruction, the `destination` account—where tokens are sent from the treasury—is not validated to ensure it belongs to the intended user. While the instruction assumes an admin will supply correct parameters, the absence of a constraint binding the `destination` account to the rightful recipient (e.g., via `user.player_id.key()`) opens a critical gap. If the admin provides an incorrect address, funds could be diverted to unintended recipients without on-chain detection.

This could lead to loss of funds for the user.

## — RECOMMENDATION

Consider enforcing that the `destination` token account owner matches the user's authority to reduce the risk of misrouted withdrawals.

## — RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

# Unclear Web2 Integration with Solana

|          |               |
|----------|---------------|
| CATEGORY | Informational |
| LOCATION | Global        |
| REVIEW   | Main Review   |

## — DESCRIPTION

The Solana programs were introduced as an alternative chain for users to deposit and withdraw. However, at the time of the review, the provided Web2 infrastructure does not appear compatible with Solana's on-chain flow.

In the EVM flow, deposits are handled by a cron job that monitors USDC transfer events into the Forwarder contract and then periodically sweeps the funds into the Comptroller. User states like deposited amounts are tracked entirely off-chain.

In contrast, on Solana:

- Deposits require a two-step user action:
- Create a deposit record via `process_deposit`.
- Perform the actual USDC transfer via `transfer_to_treasury`.
- User state (e.g., `total_deposited`, `deposit_count`) is tracked on-chain, not off-chain.
- Withdrawals should directly come from treasury account.

This structural mismatch introduces the risk of inconsistencies between the on-chain and off-chain states in Solana. Due to the lack of documentation on how the frontend/backend will integrate with Solana, it is also unclear what other integration bugs might arise.

## — RECOMMENDATION

Clarify and document how the Web2 backend will correctly synchronize with Solana and other EVM chains.

## — RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

# Contract Uses Outdated Solidity Version

|          |                                                           |
|----------|-----------------------------------------------------------|
| CATEGORY | Best Practices                                            |
| LOCATION | Forwarder.sol, ForwarderFactory.sol, GinzaComptroller.sol |
| REVIEW   | Main Review                                               |

## DESCRIPTION

The contract specifies pragma solidity `0.8.26`, which is an outdated compiler version. Solidity is under active development, and newer releases often contain critical security patches, optimizations, and language enhancements. By using an older version, the contract may remain vulnerable to issues that have already been resolved in subsequent versions. Moreover, it limits access to newer compiler checks and features that improve contract safety and maintainability. This oversight may expose users and funds to avoidable risks, reduce auditability, and hinder long-term maintainability.

## RECOMMENDATION

It is recommended to upgrade to the latest stable Solidity version after reviewing compatibility and changelogs.

## RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

# Discrepancy In Withdraw Review Amount

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Warning                         |
| LOCATION | BalanceCard.tsx, withdrawals.ts |
| REVIEW   | Main Review                     |

## DESCRIPTION

If a user's withdraw amount is exactly \$25,000, the frontend will indicate that the withdrawal request was submitted (and not under review). See `BalanceCard.tsx: 301`:

```
const requiresReview = auditResult.result === "FAILED" || withdrawalAmountNumber > 25000;
```

However, in `withdrawals.ts`, convex will mark the withdrawal as `UNDER_REVIEW` and will not process the withdrawal:

```
const status =  
auditResult === "FAILED" || withdrawAmount >= REVIEW_THRESHOLD_MICRODOLLARS  
? "UNDER_REVIEW"  
: "PENDING";
```

This discrepancy is unexpected and can cause confusion for users.

## RECOMMENDATION

In `BalanceCard.tsx`, amend the check to be greater or equal than:

```
const requiresReview = auditResult.result === "FAILED" || withdrawalAmountNumber >= 25000;
```

## RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

# tx\_signature is not used

|          |                    |
|----------|--------------------|
| CATEGORY | Best Practices     |
| LOCATION | process_deposit.rs |
| REVIEW   | Main Review        |

## — DESCRIPTION

When calling `process_signature`, the user passes a `tx_signature` which is saved in the deposit account, but is never verified.

## — RECOMMENDATION

Consider utilizing the signature or removing it if it's not needed.

## — RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

# Game States Are Not Truly Random

CATEGORY Unexpected Behavior  
LOCATION General  
REVIEW Main Review

## — DESCRIPTION

The `shuffle_and_encrypt` and `shuffle_and_commit_deck` functions rely on Python's default `random.shuffle` (MT19937), which is not cryptographically secure.

An attacker who can observe enough outputs, or guess the seed—can reconstruct the PRNG state and predict future shuffles, completely breaking game fairness. While this attack is clearly theoretical, in the past, multiple occurrences of RNG-related exploits were used in the wild, so this concern should be taken into account during system architecture hardening.

## — RECOMMENDATION

Replace all uses of `random.shuffle` with a cryptographically secure alternative, such as Python's `secrets` library.

## — RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

## Incorrect account spaces

|          |                |
|----------|----------------|
| CATEGORY | Best Practices |
| LOCATION | constants.rs   |
| REVIEW   | Main Review    |

### — DESCRIPTION

In `constants.rs`, there are minor inconsistencies in how space calculations are handled:

- `CONFIG_ACCOUNT_SIZE` appears to omit an extra byte for the bump
- Meanwhile, `USER_ACCOUNT_SIZE` includes space for a bump, but the `User` struct does not actually store a bump field

Furthermore, these constants are declared but not referenced anywhere in the codebase.

### — RECOMMENDATION

Review how bumps are handled in account size calculations, and consider removing these constants if they are unused.

### — RESOLUTION

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

# Deposits can be processed by anyone

|          |                         |
|----------|-------------------------|
| CATEGORY | Access Control          |
| LOCATION | transfer_to_treasury.rs |
| REVIEW   | Main Review             |

## — DESCRIPTION

The `transfer_to_treasury` function can be called by anyone on behalf of another user's deposit. While no direct security risks were identified with this approach, it introduces the potential for unexpected behavior for users and off-chain systems. For example, users might find their deposits transferred without their explicit action, which could cause confusion or discrepancies in backend accounting and user-facing balances.

## — RECOMMENDATION

Consider restricting `transfer_to_treasury` so that only the owner of the deposit can perform the transfer. Alternatively, if the current design is intentional, explicitly document this behavior for users and integrators to avoid surprises.

## — RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

# ATA is not initialized on config update

CATEGORY Informational  
LOCATION update\_config.rs  
REVIEW Main Review

## — DESCRIPTION

When the program is initialized, the `treasury_token_account` is initialized for the particular `token_mint` and `treasury`. The `update_config` instruction allows changing both the `treasury` and `token_mint`, but it doesn't initialize the ATA for them. In result the program may be interacting with an account that's not initialized leading to potential execution failures.

## — RECOMMENDATION

Use `init_if_needed` to initialize a new ATA when changing the config.

## — RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

# EVM Withdrawals may fail

|          |                     |
|----------|---------------------|
| CATEGORY | Unexpected Behavior |
| LOCATION | Global              |
| REVIEW   | Main Review         |

## DESCRIPTION

As we can see in `crons.ts`, deposits are processed each 10 seconds, but the funds deposited are flushed once every 30 minutes. Because of this, there is a period of time where withdrawals may be failing due to `GinzaComptroller` not having enough funds to pay them out. There is a `retryWithdrawal()` function in `withdrawals.tx`, but it's never used in the codebase.

## RECOMMENDATION

Make sure to warn users for this scenario and implement a proper retry mechanism.

## RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

## PnL is subtracted from total deposited

CATEGORY Informational  
LOCATION withdraw.rs  
REVIEW Main Review

### — DESCRIPTION

The `amount` used in the `withdraw` instruction can include positive PnL and therefore be greater than `user.total_deposited`. The code will subtract the whole amount from `user.total_deposited` and in result `total_deposited` won't accurately reflect the net amount deposited by the user.

### — RECOMMENDATION

Consider if subtracting the whole amount is desirable.

### — RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

# Missing toolchain version in Anchor.toml

|          |                |
|----------|----------------|
| CATEGORY | Best Practices |
| LOCATION | Anchor.toml    |
| REVIEW   | Main Review    |

## DESCRIPTION

The `Anchor.toml` file lacks explicit `anchor_version` and `solana_version` declarations under the `[toolchain]` section. This omission can lead to version drift across different development environments, potentially introducing unexpected behavior, compilation errors, or inconsistencies during deployment. Without a fixed toolchain version, CI/CD pipelines and team members may inadvertently use incompatible versions of Anchor or Solana.

## RECOMMENDATION

Explicitly define the Anchor and Solana versions in the `[toolchain]` section of `Anchor.toml`.

## RESOLUTION

Status: Acknowledged.

LOW **L-21**

ACKNOWLEDGED

# Deposit Account Can Be Closed To Get Rent Refund

CATEGORY Best Practices

LOCATION TransferToTreasury.rs:L29

REVIEW Main Review

## — DESCRIPTION

After a successful token transfer in the `withdraw` instruction, the corresponding `Deposit` account remains open despite having served its purpose. Since the `Deposit` account was created and rent-paid by the `user` during the `ProcessDeposit` instruction, retaining the account unnecessarily locks their SOL. This results in inefficient rent usage and user dissatisfaction due to unreclaimed funds.

## — RECOMMENDATION

Close the `Deposit` account after marking it processed and refund the rent to the original payer.

## — RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

# Flushed DepositStatus Unused

|          |             |
|----------|-------------|
| CATEGORY | Warning     |
| LOCATION | deposits.ts |
| REVIEW   | Main Review |

## — DESCRIPTION

When reconciling a deposit, `existingDeposit.status === "FLUSHED"` is performed to check if the deposit stored in the DB is FLUSHED, and if so, there is no need to credit the user again. However, `DepositStatus.FLUSHED` is never attributed to a deposit within the codebase, hence the check is redundant. Furthermore, because `DepositStatus.FLUSHED` is never used, this means the translated status from `translateStatus` will never be "REQUIRES\_MANUAL\_RECONCILIATION".

## — RECOMMENDATION

Remove the `existingDeposit.status === "FLUSHED"` case from `reconcileSingleDeposit` and clarify the flow for manual reconciliation.

## — RESOLUTION

Status: Acknowledged.

LOW **L-23**

ACKNOWLEDGED

# Unfinished development

|          |                                         |
|----------|-----------------------------------------|
| CATEGORY | Best Practices                          |
| LOCATION | apps/frontend/utils/auditUserBalance.ts |
| REVIEW   | Main Review                             |

## DESCRIPTION

The function `logAuditResults` is not implemented. It may indicate an unfinished development, or a leftover that should be removed to improve maintainability of the code.

```
export async function logAuditResults(auditResult: Awaited<ReturnType<typeof  
auditUserBalance>>) {  
  // TODO: add audit table later to track audit results / track failures/ flag suspicious  
  accounts //@note todo comment  
  return auditResult;  
}
```

## RECOMMENDATION

Remove the function or implement its logic.

## RESOLUTION

Status: Acknowledged.

LOW **L-24**

ACKNOWLEDGED

# Cross-Chain Bridge Not Properly Supported

CATEGORY Warning  
LOCATION withdrawals.ts  
REVIEW Main Review

## — DESCRIPTION

For cross-chain withdrawals, an api call to `https://api.relay.link/quote` is performed. According to the API documentation, "[w]hen executing orders using the API directly, there are often multiple steps, like submitting a deposit transaction to the solver or signing a message. These steps differ based on the desired action and best route to execute the action."

Currently Ginza's `/get_withdrawal_quote` endpoint simply retrieves the first step returned by the Relay API to find the deposit address to transfer user funds for the relay. Ginza is assuming there will only be a single step, when there can technically be multiple that have to be performed to transfer the USDC cross-chain.

## — RECOMMENDATION

Gracefully handle the case where there is more than one step.

## — RESOLUTION

Status: Acknowledged.

LOW **L-25**

ACKNOWLEDGED

# Lack of Address Validation

|          |                                       |
|----------|---------------------------------------|
| CATEGORY | Best Practices                        |
| LOCATION | packages/convex/convex/withdrawals.ts |
| REVIEW   | Main Review                           |

## DESCRIPTION

There's no validation of the `withdrawAddress` format. An invalid or malformed address provided by an user could lead to permanent loss of funds.

```
export const requestWithdrawal = userAuthenticatedMutation({
  args: {
    withdrawAddress: v.string(),
    withdrawAmount: v.number(),
    tokenContractAddress: v.string(),
    chainId: v.number(),
    auditResult: v.string(),
  },
  [...]
})
```

## RECOMMENDATION

Implement strict validation of blockchain addresses including checksums and format verification specific to the target blockchain.

## RESOLUTION

Status: Acknowledged.

LOW **L-26**

ACKNOWLEDGED

## retryWithdrawal Not Callable

|          |                |
|----------|----------------|
| CATEGORY | Warning        |
| LOCATION | withdrawals.ts |
| REVIEW   | Main Review    |

### — DESCRIPTION

Function `retryWithdrawal` is an `internalMutation`, hence it is only callable by other Convex functions. However, no Conex functions call this mutation, hence the function is never used. Additionally, `retryWithdrawal` will attempt to process the withdrawal regardless if the withdrawal is `UNDER_REVIEW` or `REJECTED`.

### — RECOMMENDATION

Consider having another admin authentication mutation to call `retryWithdrawal`, or remove it.

### — RESOLUTION

Status: Acknowledged.

LOW **L-27**

ACKNOWLEDGED

# Unvalidated patch of arbitrary gameData

CATEGORY Validation

LOCATION packages/convex/convex/tasks.ts – `updateGameData` and `updateGameDataAndTimeout`

REVIEW Main Review

## DESCRIPTION

It was found that the `updateGameData` and `updateGameDataAndTimeout` mutations accept free-form JSON under `gameData` from the game server and immediately patch it into the DB:

```
await ctx.db.patch(gameId, { gameData, updatedAt: Date.now() });
```

Because `gameData` is typed as `v.any()`, a compromised server could inject unexpected or malicious fields and data into the database, bypassing any invariants.

## RECOMMENDATION

We recommend defining a strict schema for `gameData` and validate every field properly. Reject or strip unknown keys and enforce type and size limits before persisting.

## RESOLUTION

Status: Acknowledged.

LOW **L-28**

ACKNOWLEDGED

# Unclear Function Naming

CATEGORY Best Practices  
LOCATION withdrawals.ts  
REVIEW Main Review

## DESCRIPTION

Function `getUserPendingWithdrawals` returns not only withdrawals with status "PENDING", but also withdrawals with status "UNDER\_REVIEW".

## RECOMMENDATION

Consider renaming the function or documenting this.

## RESOLUTION

Status: Acknowledged.

LOW **L-29**

ACKNOWLEDGED

# Unhandled exceptions in webhook routes

|          |                                                                      |
|----------|----------------------------------------------------------------------|
| CATEGORY | Configuration                                                        |
| LOCATION | packages/convex/convex/http.ts – `/withdrawal_confirmation` endpoint |
| REVIEW   | Main Review                                                          |

## — DESCRIPTION

It was found, that an error thrown by `validateThirdWebWebhook` results in a server response with potential stack trace leakage. This can disrupt the HTTP router and expose internal details to external callers.

It is a good security practice to return just a generic error message with, for example, parameters used, not the whole `error` object to the caller.

## — RECOMMENDATION

We recommend wrapping each handler in a `try/catch` block. On error, log details server-side and return a controlled 400 (or 401) with a generic message.

## — RESOLUTION

Status: Acknowledged.

LOW **L-30**

ACKNOWLEDGED

# Missing HTTP Security Headers

CATEGORY Configuration  
LOCATION [www.ginzagaming.com](http://www.ginzagaming.com)  
REVIEW Main Review

## DESCRIPTION

Missing security headers can lead to multiple attack vectors:

- No X-Frame-Options may allow clickjacking attacks, tricking users into clicking hidden elements.
- No X-Content-Type-Options could enable MIME-type sniffing, allowing the browser to execute malicious files incorrectly.
- No Content-Security-Policy allows the browser to accept potentially dangerous inline scripts or external resources, increasing the risk of XSS.

## RECOMMENDATION

It is recommended to include the following HTTP response headers to enhance client-side security:

- X-Frame-Options: DENY or SAMEORIGIN to prevent clickjacking.
- X-Content-Type-Options: nosniff to stop MIME-type sniffing.
- Content-Security-Policy with appropriate directives to limit the sources of executable scripts, styles, and frames.

## RESOLUTION

Status: Acknowledged.

LOW **L-31**

ACKNOWLEDGED

# Lack of Server Side Validation

CATEGORY Trust Assumptions  
LOCATION clerk.ginzagaming.com  
REVIEW Main Review

## DESCRIPTION

In Next.js applications using the @clerk/nextjs SDK, authentication is typically handled through JWTs (JSON Web Tokens). To optimize performance, the SDK verifies the JWT once in the middleware and then passes the authentication state to the endpoint handler via a custom header. However, due to a refactoring error introduced in version 4.7.0, the endpoint handler began prioritizing the JWT from cookies over the header. This discrepancy allowed attackers to craft a scenario where the middleware would validate a legitimate JWT in the header, but the endpoint handler would process a malicious JWT from the cookie without re-verification. This flaw enabled attackers to:

- Act on behalf of other users: By modifying the sub claim in the JWT, attackers could impersonate any user.
- Escalate privileges: By altering claims like role, attackers could gain unauthorized access to restricted functionalities.

The above screenshot shows that when we are changing the clerk\_js\_version to 4.7.0 it is giving a success response instead of throwing an error.

## RECOMMENDATION

Implement the checks for clerk\_js\_version being used.

## RESOLUTION

Status: Acknowledged.

## § 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**DIGITAL ASSETS UNDER  
REVIEW**200+**

ENGAGEMENTS COMPLETED

**500+**CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).